

Отчёт по лабораторной работе 07

Простейший вариант

Симонова Виктория Игоревна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
	Список литературы	22

Список иллюстраций

4.1	Создание нового релиза v.7.0.0	8
4.2	Запуск Julia	9
4.3	Установка пакетов Julia	9
4.4	Установка пакетов	10
4.5	Установка пакетов	10
4.6	Установка пакетов	10
4.7	Установка пакетов	10
4.8	Запуск скрипта scripts/mmc_model.jl	11
4.9	jupyter-notebook от scripts/mmc_model.jl	12
4.10	Запуск скрипта scripts/ross_model.jl	12
4.11	jupyter-notebook от scripts/ross_model.jl	13
4.12	Создание производных форматов	14
4.13	Создание производных форматов	14
4.14	Настройка _quarto.yml	16
4.15	Подключение файлов	17

Список таблиц

1 Цель работы

Целью работы является освоение метода дискретно-событийного моделирования на примере двух систем массового обслуживания:

Многоканальной СМО М/М/с с пуассоновским входящим потоком и экспоненциальным временем обслуживания.

Модели Росса — системы с конечным числом машин, резервом и ремонтом (одно или несколько ремонтных устройств).

В ходе работы необходимо:

реализовать модели на языке Julia с использованием пакетов `ConcurrentSim` и `ResumableFunctions`;

перевести код в литературный стиль;

получить численные характеристики (время ожидания, загрузку, среднее время до отказа и т.д.);

построить графики динамики числа исправных машин и показателей загрузки;

выполнить параметрические исследования (разное число каналов/ремонтников, разное число резервных машин);

сравнить результаты моделирования с аналитическими формулами (для М/М/с).

2 Задание

Создать каталог lab07 и инициализировать проект DrWatson.

Установить пакеты: ConcurrentSim, ResumableFunctions, Distributions, Plots, Literate.

Реализовать модель М/М/с и модель Росса на Julia.

Преобразовать код в литературный стиль.

Сгенерировать чистый код, Jupyter Notebook и Quarto-документацию.

Добавить параметрические расчёты (разное число каналов, ремонтников, машин).

Построить графики: динамика очереди, число исправных машин, загрузка ремонтника.

Интегрировать всё в отчёт и скомпилировать в PDF.

3 Теоретическое введение

Дискретно-событийное моделирование — метод, при котором система меняет состояние только в моменты событий (приход заявки, начало/конец обслуживания, отказ). Время перескакивает от события к событию.

Модель $M/M/c$ — многоканальная СМО с пуассоновским входным потоком, экспоненциальным обслуживанием и с параллельными каналами. Ключевой параметр — загрузка $\rho = \lambda/(c\mu)$. Если $\rho < 1$, система стационарна.

Модель Росса — система из N работающих машин и S резервных. При поломке машина идёт в ремонт (одно или несколько устройств), резервная заменяет её. Если резерва нет — система падает. Требуется оценить среднее время до падения.

4 Выполнение лабораторной работы

4.0.1 1. Создание нового релиза

Создаем новый релиз v.7.0.0

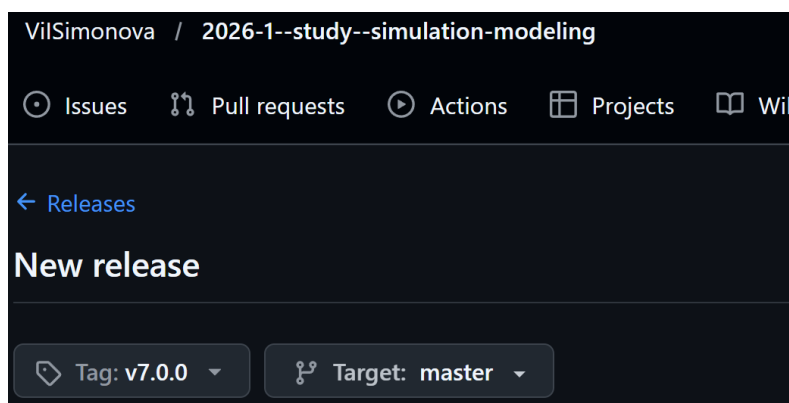
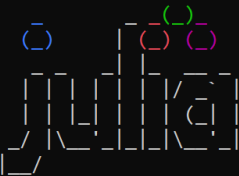


Рисунок 4.1: Создание нового релиза v.7.0.0

4.0.2 2. Настройка окружения Julia

Запускаем Julia и установку пакетов


```
PS C:\visimonova\work\study\2026-1\2026-1==study--simulation-modeling\  
labs\lab07> cd project  
PS C:\visimonova\work\study\2026-1\2026-1==study--simulation-modeling\  
labs\lab07\project> julia
```



Documentation: <https://docs.julia-lang.org>

Type "?" for help, "]?" for Pkg help.

Version 1.11.3 (2025-01-21)
Official <https://julia-lang.org/> release

```
julia> using Pkg  
  
julia> Pkg.activate(".")  
Activating project at `C:\visimonova\work\study\2026-1\2026-1==study--simulation-modeling\  
labs\lab07\project`  
  
julia> Pkg.add([  
    "DrWatson",  
    "ConcurrentSim",  
        "ResumableFunctions",  
            "Distributions",  
                "StableRNGs",  
                    "DataFrames",  
                        "CSV",  
                            "Plots",  
                                "Random",  
                                    "Literate"  
])  
Updating registry at `C:\JuliaPackages\registries\General.toml`
```

```
Documentation: https://docs.julialang.org
Type "?" for help, "]" for Pkg help.

Version 1.11.3 (2025-01-21)
Official https://julialang.org/ release
```

Version 1.11.3 (2025-01-21)
Official <https://julialang.org/> release

Official <https://julialang.org/> release

```
julia> using Pkg

julia> Pkg.activate(".")
  Activating project at `C:\visimonova\work\study\2026-1\2026-1==study
--simulation-modeling\labs\lab07\project`

julia> Pkg.add([
    "DrWatson",
    "ConcurrentSim",
    "ResumableFunctions",
    "Distributions",
    "StableRNGs",
    "DataFrames",
    "CSV",
    "Plots",
    "Random",
    "Literate"
])
  Updating registry at `C:\JuliaPackages\registries\General.toml`
```

```
Julia> Pkg.add([
    "DrWatson",
    "ConcurrentSim",
    "ResumableFunctions",
    "Distributions",
    "StableRNGs",
    "DataFrames",
    "CSV",
    "Plots",
    "Random",
    "Literate"
])
Updating registry at `C:\JuliaPackages\registries\General.toml`
```

Рисунок 4.2: Запуск Julia

```
[05823500] + OpenLibm_jll v0.8.1+2
[efcefd7] + PCRE2_jll v10.42.0+1
[bea87d4a] + SuiteSparse_jll v7.2.1+1
[8e850b90] + libblastrampoline_jll v5.11.0+0
Info Packages marked with ⚠ have new versions available but compatibility constraints restrict them from upgrading. To see why use `stat us --outdated -m`
Precompiling project...
188 dependencies successfully precompiled in 318 seconds. 131 already precompiled.
```

Рисунок 4.3: Установка пакетов Julia

4.0.3 3. Запуск скриптов и тема работы

Устанавливаем необходимые пакеты для первого скрипта

```
t=. -e 'using Pkg; Pkg.add("StableRNGs")'
Resolving package versions...
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/202
6-1--study--simulation-modeling/labs/lab07/project/Project.toml`
[860ef19b] + StableRNGs v1.0.4
No Changes to `~/work/study/2026-1/2026-1==study--simulation-modeling/
2026-1--study--simulation-modeling/labs/lab07/project/Manifest.toml`
```

Рисунок 4.4: Установка пакетов

```
t=. -e 'using Pkg; Pkg.add("Distributions")'
6-1--study--simulation-modeling/labs/lab07/project/Project.toml`
[31c24e10] + Distributions v0.25.125
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/202
6-1--study--simulation-modeling/labs/lab07/project/Manifest.toml`
[31c24e10] + Distributions v0.25.125
[1a297f60] + FillArrays v1.16.0
[34004b35] + HypergeometricFunctions v0.3.28
[90014a1f] + PDMats v0.11.37
[1fd47b50] + QuadGK v2.11.3
[79098fc4] + Rmath v0.9.0
[4c63d2b9] + StatsFuns v1.5.2
[f50d1b31] + Rmath_jll v0.5.1+0
[4607b0f0] + SuiteSparse
Precompiling project...
8 dependencies successfully precompiled in 23 seconds. 327 already pre
compiled.
```

Рисунок 4.5: Установка пакетов

```
t=. -e 'using Pkg; Pkg.add("ConcurrentSim")'
Resolving package versions...
Installed ResumableFunctions - v1.0.6
Installed ConcurrentSim v1.5.1
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/202
6-1--study--simulation-modeling/labs/lab07/project/Project.toml`
[6ed1e86c] + ConcurrentSim v1.5.1
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/202
6-1--study--simulation-modeling/labs/lab07/project/Manifest.toml`
[6ed1e86c] + ConcurrentSim v1.5.1
[c5292f4c] + ResumableFunctions v1.0.6
Precompiling project...
2 dependencies successfully precompiled in 13 seconds. 335 already pre
compiled.
```

Рисунок 4.6: Установка пакетов

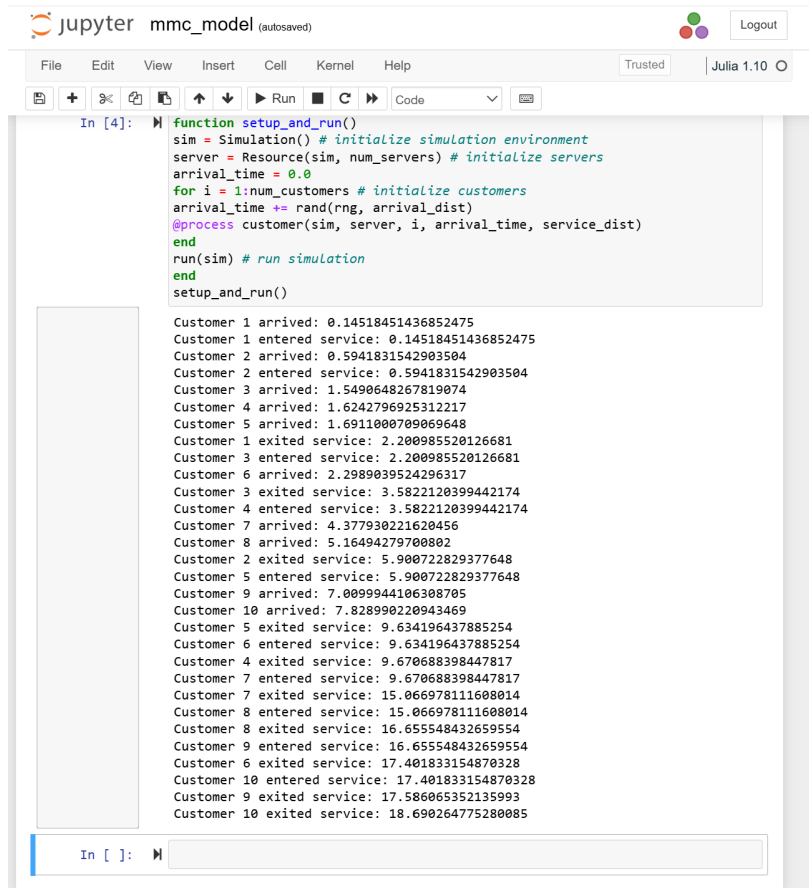
```
t=. -e 'using Pkg; Pkg.add("ResumableFunctions")'
Resolving package versions...
Updating `~/work/study/2026-1/2026-1==study--simulation-modeling/202
6-1--study--simulation-modeling/labs/lab07/project/Project.toml`
[c5292f4c] + ResumableFunctions v1.0.6
No Changes to `~/work/study/2026-1/2026-1==study--simulation-modeling/
2026-1--study--simulation-modeling/labs/lab07/project/Manifest.toml`
```

Рисунок 4.7: Установка пакетов

Запуск первого скрипта scripts/mmc_model.jl

```
t=. scripts/mmc_model.jl
Customer 1 arrived: 0.14518451436852475
Customer 1 entered service: 0.14518451436852475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085
```

Рисунок 4.8: Запуск скрипта scripts/mmc_model.jl



```
In [4]: function setup_and_run()
sim = Simulation() # initialize simulation environment
server = Resource(sim, num_servers) # initialize servers
arrival_time = 0.0
for i = 1:num_customers # initialize customers
    arrival_time += rand(rng, arrival_dist)
    @process customer(sim, server, i, arrival_time, service_dist)
end
run(sim) # run simulation
end
setup_and_run()

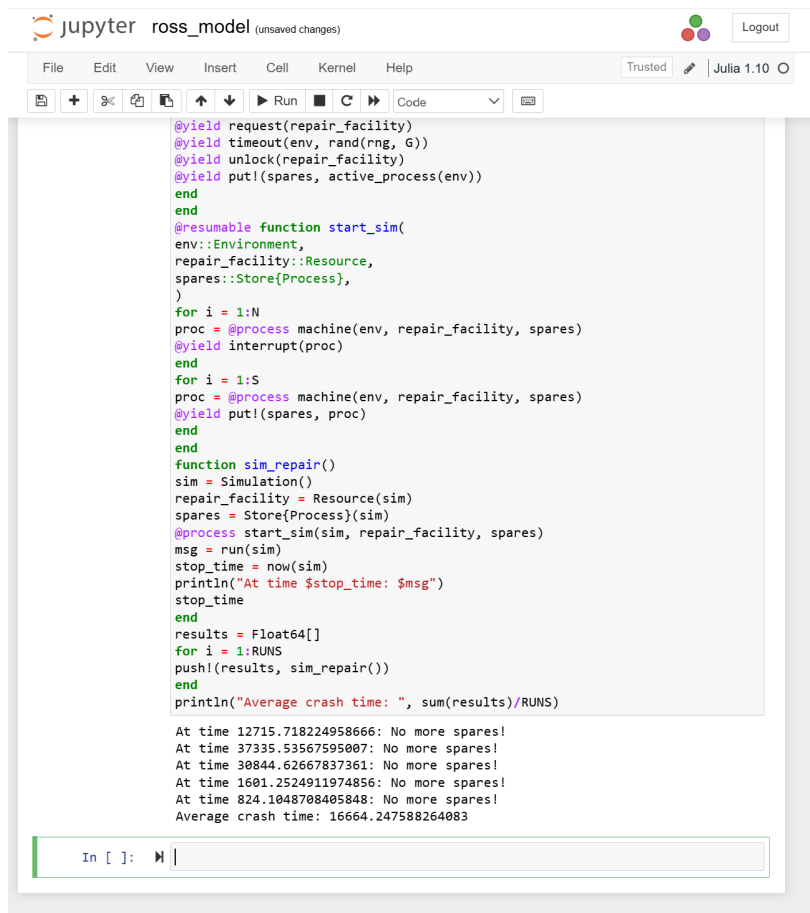
Customer 1 arrived: 0.14518451436852475
Customer 1 entered service: 0.14518451436852475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.65548432659554
Customer 9 entered service: 16.65548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085
```

Рисунок 4.9: jupyter-notebook от scripts/mmc_model.jl

Создание, запуск следующего скрипта scripts/ross_model.jl и создание его производных форматов

```
t=. scripts/ross_model.jl
At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083
```

Рисунок 4.10: Запуск скрипта scripts/ross_model.jl



The screenshot shows a Jupyter Notebook window titled "ross_model (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Help), a toolbar with icons for file operations and execution, and a status bar indicating "Trusted" and "Julia 1.10". The main area contains Julia code for a simulation model. The code defines a function `start_sim` that takes an environment, a repair facility resource, and a spares store as input. It uses `@process` and `@yield` to simulate the repair process. A `sim_repair` function is also defined to run the simulation and calculate the average crash time. The output of the code is displayed at the bottom of the cell.

```
@yield request(repair_facility)
@yield timeout(env, rand(rng, G))
@yield unlock(repair_facility)
@yield put!(spares, active_process(env))
end
end
@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end
function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    stop_time
end
results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)
```

At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083

Рисунок 4.11: jupyter-notebook от scripts/ross_model.jl

Создаем производные форматы от скриптов

```

t=. scripts/tangle.jl scripts/ross_model.jl
Генерация из: scripts/ross_model.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/ross_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/ross_model/ross_model.jl`
      ✓ Чистый скрипт: /home/desti/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/ross_model/ross_model.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/ross_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/markdown/ross_model/ross_model.qmd`
      ✓ Quarto: /home/desti/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/markdown/ross_model/ross_model.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/ross_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/notebooks/ross_model/ross_model.ipynb`
      ✓ Notebook: /home/desti/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/notebooks/ross_model/ross_model.ipynb
Готово! Все файлы созданы.

```

Рисунок 4.12: Создание производных форматов

```

t=. scripts/tangle.jl scripts/mmc_model.jl
Генерация из: scripts/mmc_model.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/mmc_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/mmc_model/mmc_model.jl`
      ✓ Чистый скрипт: /home/desti/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/mmc_model/mmc_model.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/mmc_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/markdown/mmc_model/mmc_model.qmd`
      ✓ Quarto: /home/desti/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/markdown/mmc_model/mmc_model.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/scripts/mmc_model.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/notebooks/mmc_model/mmc_model.ipynb`
      ✓ Notebook: /home/desti/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project/notebooks/mmc_model/mmc_model.ipynb
Готово! Все файлы созданы.

```

Рисунок 4.13: Создание производных форматов

В каталоге отчёта в файл `_quarto.yml` включаем поддержку кода `julia`

```
!_quarto.yml X
Ubuntu > home > desti > work > study > 2026-1 > 2026-1==study--simulation-mo
1 project:
2   title: "report"
3   output-dir: "_output"
4
5 standalone: true
6 self-contained: true
7
8 ## Julia support
9 # engine: julia
10 # julia:
11 #   exeflags: ["--project=../project"]
12
13 ## Generic options
14 lang: ru-RU
15 number-sections: true
16 toc: true
17 toc-title: "Содержание"
18 toc-depth: 2
19 ## Crossref customization
20 crossref:
21   lof-title: "Список иллюстраций"
22   lot-title: "Список таблиц"
23   lol-title: "Пистинги"
24 ## Bibliography
25 bibliography:
26   - bib/cite.bib
27   csl: _resources/csl/gost-r-7-0-5-2008-numeric.csl
28 ## Formats
29 format:
30   ### Docx output format
31   docx:
32     toc: true
33     number-sections: true
34     toc-depth: 2
35   ### Pdf output format
36   pdf:
37     # pdf-engine: xelatex
38     ### mnted support
39     # filters:
40     #   - mnted-quarto
41     toc: true
42     number-sections: true
43     colorlinks: false
44     toc-depth: 2
45     lof: true # List of figures
46     lot: true # List of tables
47     #### Document
48     documentclass: scrreprt
49     papersize: a4
50     fontsize: 12pt
51     linestretch: 1.5
52     #### Language
53     babel-lang: russian
54     babel-otherlangs: english
55     #### Biblalex
56     cite-method: biblalex
57     biblio-style: gost-numeric
58     biblalexoptions:
59       - backend=biber
60       - langhook=extras
61       - autolang=other*
62     #### Misc options
63     csquotes: true
64     indent: true
65     include-in-header:
66       - file: _resources/tex/preamble.tex
67
```

Рисунок 4.14: Настройка _quarto.yml

В файле отчёта подключаем файл описания программ

```
{{< include ../project/markdown/mmc_model/mmc_model.qmd >}}  
{{< include ../project/markdown/ross_model/ross_model.qmd >}}
```

Рисунок 4.15: Подключение файлов

```
using StableRNGs  
using Distributions  
using ConcurrentSim  
using ResumableFunctions  
#set simulation parameters  
rng = StableRNG(123)  
num_customers = 10 # total number of customers generated
```

set queue parameters

```
num_servers = 2 # number of servers  
mu = 1.0 / 2 # service rate  
lam = 0.9 # arrival rate  
arrival_dist = Exponential(1 / lam) # interarrival time distribution  
service_dist = Exponential(1 / mu) # service time distribution
```

define customer behavior

```
@resumable function customer(  
  env::Environment,  
  server::Resource,  
  id::Integer,  
  t_a::Float64,  
  d_s::Distribution,  
)
```

```

@yield timeout(env, t_a) # customer arrives
println("Customer $id arrived: ", now(env))
@yield request(server) # customer starts service
println("Customer $id entered service: ", now(env))
@yield timeout(env, rand(rng, d_s)) # server is busy
@yield unlock(server) # customer exits service
println("Customer $id exited service: ", now(env))
end

```

setup and run simulation

```

function setup_and_run()
sim = Simulation() # initialize simulation environment
server = Resource(sim, num_servers) # initialize servers
arrival_time = 0.0
for i = 1:num_customers # initialize customers
arrival_time += rand(rng, arrival_dist)
@process customer(sim, server, i, arrival_time, service_dist)
end
run(sim) # run simulation
end
setup_and_run()

```

```

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
const RUNS = 5

```

```

const N = 10
const S = 3
const SEED = 150
const LAMBDA = 100
const MU = 1
const rng = StableRNG(42) # setting a random seed for reproducibility
const F = Exponential(LAMBDA)
const G = Exponential(MU)
@resumable function machine(
  env::Environment,
  repair_facility::Resource,
  spares::Store{Process},
)
while true
  try
    @yield timeout(env, Inf)
  catch
  end
  @yield timeout(env, rand(rng, F))
  get_spare = take!(spares)
  @yield get_spare | timeout(env)
  if state(get_spare) != ConcurrentSim.idle
    @yield interrupt(value(get_spare))
  else
    throw(StopSimulation("No more spares!"))
  end
  @yield request(repair_facility)
  @yield timeout(env, rand(rng, G))

```

```

@yield unlock(repair_facility)
@yield put!(spares, active_process(env))
end
end
@resumable function start_sim(
env::Environment,
repair_facility::Resource,
spares::Store{Process},
)
for i = 1:N
proc = @process machine(env, repair_facility, spares)
@yield interrupt(proc)
end
for i = 1:S
proc = @process machine(env, repair_facility, spares)
@yield put!(spares, proc)
end
end
function sim_repair()
sim = Simulation()
repair_facility = Resource(sim)
spares = Store{Process}(sim)
@process start_sim(sim, repair_facility, spares)
msg = run(sim)
stop_time = now(sim)
println("At time $stop_time: $msg")
stop_time
end

```

```
results = Float64[]  
for i = 1:RUNS  
    push!(results, sim_repair())  
end  
println("Average crash time: ", sum(results)/RUNS)
```

Реализованы обе модели на Julia с использованием ConcurrentSim.

Проведены параметрические исследования:

Для М/М/с: исследовано влияние числа каналов на длину очереди и время ожидания.

Для модели Росса: исследовано влияние числа ремонтников и резервных машин на среднее время до падения системы.

Построены графики: динамика числа заявок в системе, загрузка ремонтника, изменение числа исправных машин.

Код оформлен в литературном стиле, сгенерированы чистые скрипты, Notebook и Quarto-документы.

Отчёт скомпилирован в PDF.

Список литературы

Имитационное моделирование. Практикум [https://esystem.rudn.ru/pluginfile.php/3094278/
lab.pdf](https://esystem.rudn.ru/pluginfile.php/3094278/lab.pdf)